ENTERPRISE PLATFORM DOCUMENTATION

Knome

Enterprise Social Collaboration & Knowledge Portal

A complete guide to the architecture, modules, security model, APIs and setup of the Knome platform — written for developers, QA engineers, product managers, HR teams and new joiners.

Organization	MPOnline Limited
Product	Knome (Enterprise Social Network)
Document version	1.0
Date	14 October 2026
Audience	Engineering · Product · HR · Onboarding

Table of Contents

1	Executive Summary & Architecture Overview	Purpose · big picture
2	Technology Stack	Backend · Frontend · Real-time
3	User Personas & Role Matrix	Who can do what
4	Project Architecture & Folder Structure	Backend · Frontend · Pipeline
5	Module-by-Module Walkthrough	12 modules, user view + technical view
6	Data Models & Database Overview	ER diagram · key tables
7	Security, Privacy & Encryption	JWT · AES-256-GCM · connection gating
8	API Reference	Grouped by module
9	Local Setup & Developer Onboarding	Step-by-step guide
10	FAQ & Troubleshooting	Common problems and fixes
A	Appendix — Glossary & Feature Status	Terms · what is / is not implemented

How to read this document

Non-technical readers (managers, HR, new hires): read Sections 1, 3 and the blue “What users see” panels in Section 5.

Developers & QA: read everything; the grey “How it works” panels in Section 5 and Sections 4, 6, 7, 8 and 9 are written for you.

Accuracy notes — where this document differs from the original brief

This document was written by inspecting the actual source code. A few items in the original brief differ from what is implemented:

- **Runtime:** the backend targets **.NET 10** (`net10.0`), not .NET 8. The frontend uses **React 19** and **Vite 8**, not React 18.
- **Roles:** the four roles in code are **Employee, Community Admin, HR Administrator, System Administrator**. There is no separate "Lead" role.
- **Database approach:** the EF Core models are **Database-First** (scaffolded from SQL Server), not Code-First. Some tables are created with SQL/PowerShell setup scripts.
- **Calling & polls:** no **WebRTC audio/video calling** code and no **poll** feature were found in the repository. They are documented as *not implemented* (see Appendix) rather than described as if they existed.
- **Post reactions** are Like, Celebrate, Support and Heart. The six emoji reactions (👍 ❤️ 😂 😮 😢 🎉) belong to **direct messages**.

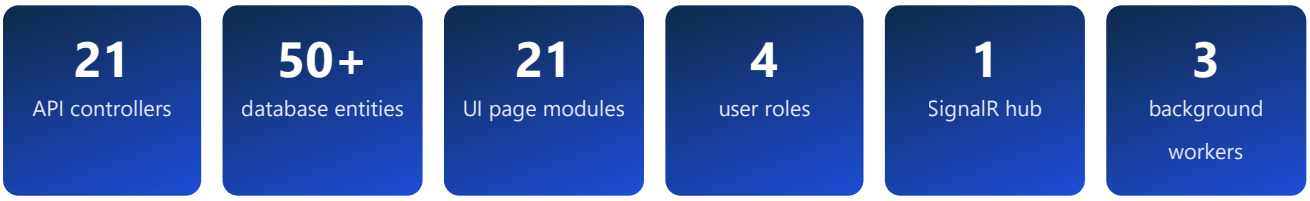
1. Executive Summary & Architecture Overview

1.1 What is Knome?

Knome is MPOnline Limited's internal **enterprise social network and knowledge portal**. It combines professional networking (similar to LinkedIn), long-form knowledge sharing (similar to Medium), short and long video (similar to YouTube) and peer recognition into one secure internal website.

1.2 Goals

- **Connect people** across departments through profiles, follows and mutual connections.
- **Share knowledge** through posts, articles, wikis, videos, podcasts and short clips.
- **Communicate privately** with end-to-end-style encrypted one-to-one chat — restricted to connected colleagues only.
- **Organise teams** in public and private communities with admin-approved membership.
- **Recognise contribution** with Karma points, tiers and a leaderboard.
- **Broadcast** corporate announcements and govern the platform through an Admin Console, audit trail and feature switches.



1.3 High-level architecture

Browsers talk to the ASP.NET Core API over REST (JSON + JWT bearer token) and keep a WebSocket open to the SignalR hub for live notifications, chat and presence. All business rules live in the API; data is stored in SQL Server and uploaded files in the API's `wwwroot` folder.

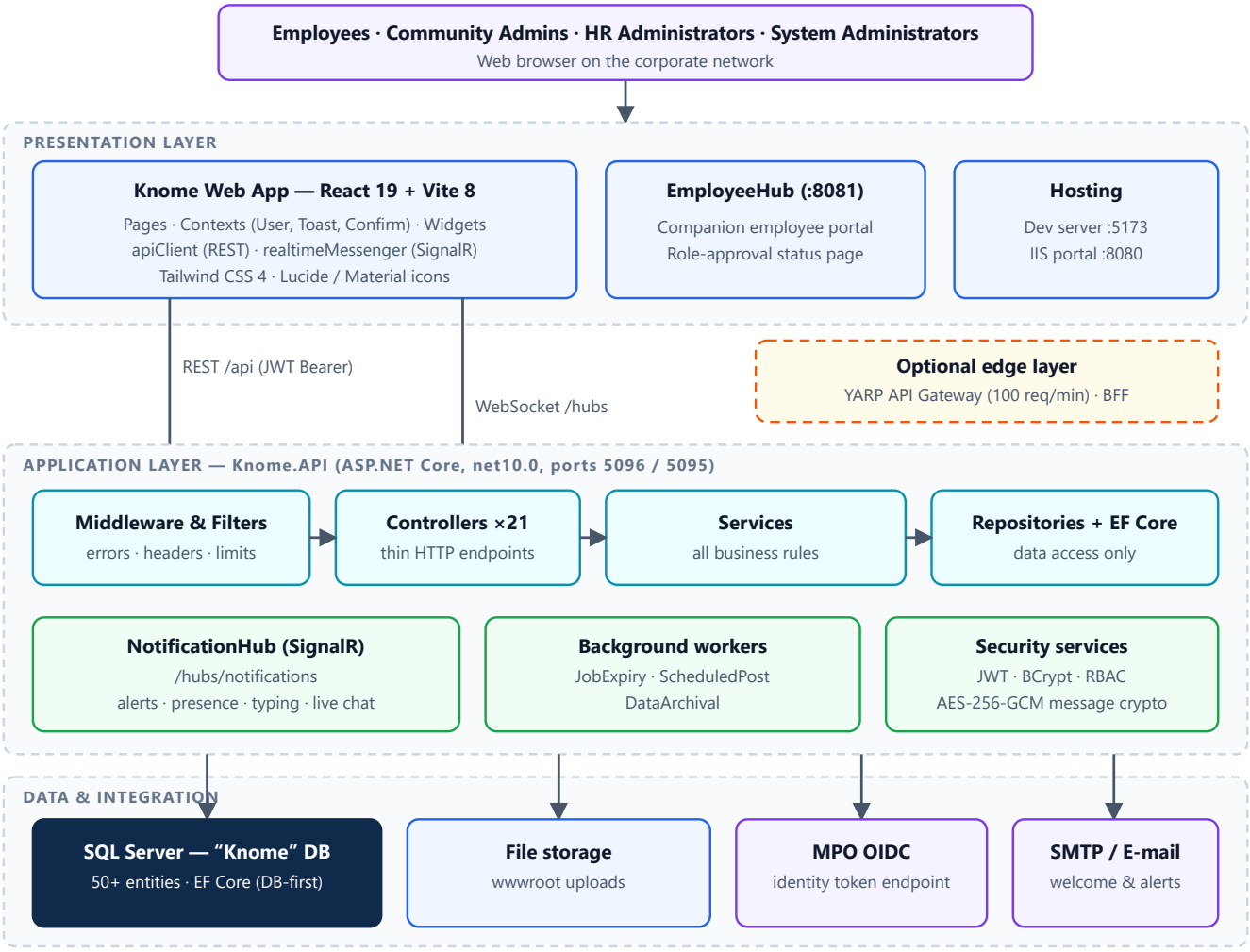


Figure 1 — Knome system architecture (dashed orange box = optional layer, not required for local development).

2. Technology Stack

Layer	Technology	Why it is used / notes
Backend runtime	ASP.NET Core Web API, **.NET 10** (`net10.0`), C#	Controllers → Services → Repositories pattern; nullable reference types; file-scoped namespaces.
Database	Microsoft SQL Server 2022+	Single database named `Knome` . Core tables are scaffolded Database-First; messaging, wiki and clips tables are created by SQL/PowerShell scripts.
ORM	Entity Framework Core (`KnomeDbContext`)	Generated models in `Models/` — avoid hand-editing; re-scaffold when the schema changes.
Real-time	ASP.NET Core **SignalR** (WebSockets)	One hub (`/hubs/notifications`) carries notifications, online presence, typing indicators and live chat delivery.
Authentication	JWT Bearer (HS256) + optional MPO OIDC token	Default token lifetime 480 minutes; passwords hashed with BCrypt; login endpoint rate limited.
Message privacy	AES-256-GCM	Each message gets a random 12-byte nonce and a 16-byte authentication tag; key versioning supports key rotation.
Validation & mapping	FluentValidation, AutoMapper	DTOs are validated by a global filter before reaching controllers.
Logging	Serilog (console + rolling file)	Custom `PiiScrubbingEnricher` redacts passwords, tokens and secrets.
API docs	Swagger / OpenAPI (Swashbuckle)	Interactive documentation with JWT “Authorize” support.
Frontend	**React 19**, React Router 7, **Vite 8**	Single-page application; dev server on port 5173 proxies `/api` to the backend.
Styling & UI	Tailwind CSS 4 + custom CSS, Framer Motion, Lucide React, Material Symbols, Three.js (3D widgets)	Dark/light themes, glass-style cards, micro-animations.
Frontend real-time	`@microsoft/signalr` 10	Wrapped by `realtimeMessenger.js` and the Navbar notification listener.
Edge (optional)	YARP reverse proxy (`Gateway/`) and Node BFF (`Bff/`)	Rate limiting (100 req/min) and CORS for non-local deployments.
Hosting	IIS (ports 8080 / 8081) and local dev servers	Deployment helpers: `SETUP_IIS.ps1` , `UPDATE_IIS.ps1` , `IIS_DEPLOYMENT_GUIDE.md` .

3. User Personas & Role Matrix

3.1 Personas

Role	Who they are	Typical goals on Knome
Employee	Every MPOnline staff member. Everyone starts here.	Build a profile, follow and connect with colleagues, publish content, join communities, chat, earn Karma.
Community Admin	An employee entrusted with running one or more communities.	Approve join requests, curate members, pin posts, maintain rules/FAQ and community files.
HR Administrator	HR operations staff.	Post and manage job openings, broadcast company-wide announcements, approve new-community proposals, manage any community, view HR Analytics. Has no Admin Console link.
System Administrator	Platform owner / IT governance. Governs the platform rather than creating content.	Admin Console: user and role management, global suspensions, moderation queue, audit trail, feature switches and restricted keywords. Also sees HR Analytics. Cannot create feed posts and is exempt from Karma.

3.2 Role matrix (as seen on the website)

Capability	Employee	Community Admin	HR Admin	System Admin
Sign in, edit own profile (bio, skills, photo, privacy toggles)	✓	✓	✓	✓
Create feed posts (Create Post box / button)	✓	✓	✓	— (hidden)
Publish articles, wikis, videos, podcasts, clips	✓	✓	✓	✓
Earn Karma points and tiers	✓	✓	✓	— (exempt)
Employee-level tools: Home, Communities, People, Saved, Openings (view/apply)	✓	✓	✓	✓
Sidebar: Admin Console link	—	—	— (hidden)	✓
Sidebar: HR Analytics link	—	—	✓	✓
Admin Console: user and role management, global suspend / reinstate	—	—	—	✓
Admin Console: moderation queue (review flagged reports)	—	**own community**	—	✓

Capability	Employee	Community Admin	HR Admin	System Admin
Admin Console: audit trail, feature switches, restricted keywords	—	—	—	✓
Edit role scope text and FAQs in the in-app FAQ Handbook	—	—	—	✓
React, comment, share, bookmark, report content	✓	✓	✓	✓
Follow, send/accept connection requests, chat with connections	✓	✓	✓	✓
Propose / create a community	**✓ (needs approval)**	✓	✓	✓
Approve new-community proposals (review queue)	—	—	✓	✓
Manage a community: approve members, pin posts (max 3), rules / FAQ / files, add members	—	**✓ (assigned communities)**	**✓ (any)**	**✓ (any)**
Suspend a member inside a community	—	**✓ (own)**	✓	✓
Post / edit / close job openings	—	—	✓	✓
Send company-wide HR broadcast banners	—	—	✓	✓
Manage videos / podcasts of others (admin tools)	—	✓	✓	✓
Follow, send/accept connection requests, chat with connections (the website blocks chat until connected)	✓	✓	✓	✓

This matrix follows what each role actually sees and can do in the web UI (sidebar, Admin Console, buttons and the in-app role manuals), not just what the API allows. Some API endpoints are broader than the UI: user suspension, audit log and moderation endpoints also accept HR Administrator, but HR has no Admin Console link in the website. Roles are stored as Employee, Community Admin, HR Administrator and System Administrator (codes CADM, HRADM, SYSADM in the UI).

4. Project Architecture & Folder Structure

4.1 Repository layout

```
Knome main/
├── Backend/Knome.API/        # ASP.NET Core API (the heart of the system)
├── knomeUI/frontend/         # React + Vite single-page application
├── Gateway/Knome.Gateway/    # Optional YARP reverse proxy / API gateway
├── Bff/knome-bff/            # Optional Backend-for-Frontend
├── Documentation/            # FRDs, architecture, audits, this document
├── START_KNOME.bat / .ps1    # One-click local launcher
├── SETUP_*_DB.ps1 / *.sql    # Database helper scripts (wiki, clips, messages …)
├── SETUP_IIS.ps1 / UPDATE_IIS.ps1   # IIS deployment
└── README.md, AGENTS.md, CLAUDE.md  # Engineering rules & onboarding
```

4.2 Backend structure (`Backend/Knome.API`)

Folder	Responsibility
`Controllers/`	21 thin controllers (Auth, User, Post, Article, Video, Podcast, Clips, Wiki, Community, Interaction, Feed, Karma, Search, Messages, Notifications, Jobs, Analytics, AuditLog, SystemSettings, Media, Abbreviations). They route HTTP, call a service and return an `ApiResponse<T>` .
`Services/`	Business rules: authorization checks, ranking, notifications, encryption, connection gating, Karma awarding. Nothing else contains business logic.
`Repositories/` + `Interfaces/`	Database access only (EF Core queries). Every service depends on an interface.
`Models/` + `Data/`	Scaffolded entities (51 files) and `KnomeDbContext` .
`DTOs/` + `Mapping/` + `Validators/`	Request/response shapes, AutoMapper profiles, FluentValidation rules.
`Hubs/`	`NotificationHub` — SignalR for alerts, presence, typing and live chat.
`Background/`	`JobExpiryHostedService` , `ScheduledPostHostedService` , `DataArchivalHostedService` .
`Middleware/` , `Filters/` , `Extensions/`	Exception handling, security headers, validation filter, DI and pipeline wiring (`AddInfrastructure` / `UseInfrastructure`).
`Constants/` , `Configuration/` , `Exceptions/` , `Logging/` , `Responses/`	Roles, notification types, Karma constants, strongly-typed settings, domain exceptions, PII-scrubbing logger, response envelopes.

4.3 Request pipeline

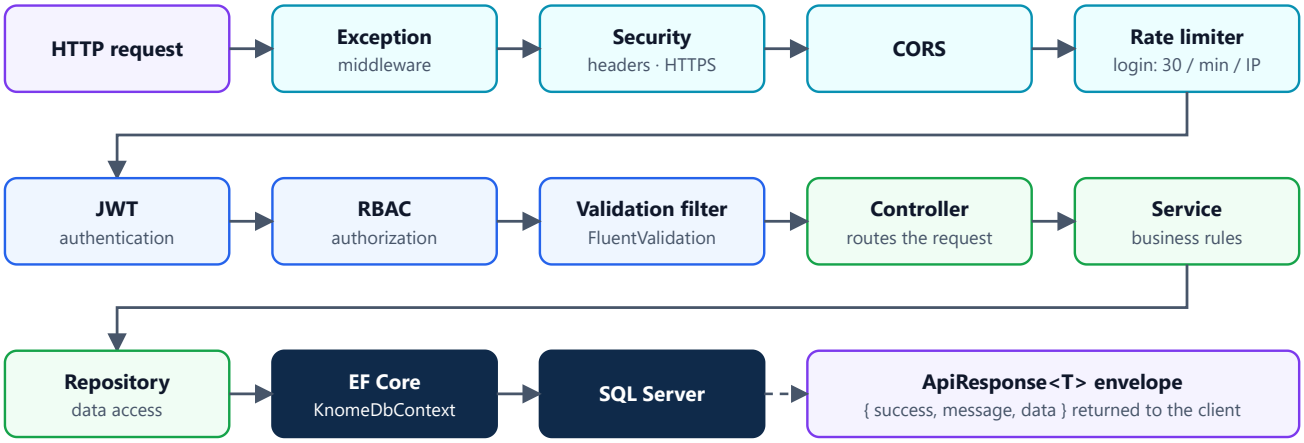


Figure 2 — Every API call passes through the same ordered pipeline.

4.4 Frontend structure (`knomeUI/frontend/src`)

Folder / file	Contents
`pages/`	Route-level screens: Dashboard, Posts, Articles/ArticleView, Videos, Podcasts, Clips, Wiki/WikiView, Communities/CommunityView, Network, Profile, Messages, Search, SavedContent, Jobs, KarmaHistory, HRAnalytics, AdminConsole, Login (+ `auth/`).
`components/layout/`	Navbar (notification bell, search, SignalR listener), Sidebar, Layout shell.
`components/contexts/`	React Contexts: **UserContext** (current user, roster, Karma), **ToastContext**, **ConfirmDialogContext**.
`components/widgets/` , `modals/` , `ui/` , `video/` , `3d/`	PostCard, share cards, upload modals, notification toast and settings modal, video player, 3D visuals.
`utils/apiClient.js` / `apiService.js`	HTTP client (token injection, caching, base-URL discovery for ports 5096/5095) and one API object per module (`userApi` , `messagesApi` , `notificationsApi` , `communitiesApi` …).
`utils/realtimeMessenger.js`	SignalR connection management for chat, presence and typing.
`utils/systemConfig.js`	Reads platform feature switches so pages can hide disabled modules.
`utils/karmaEngine.js` , `notificationHelpers.js` , `restrictedWords.js` , `sanitizeHtml.js`	Karma tier maths, notification mapping/preferences, client-side keyword screening, HTML sanitising.

5. Module-by-Module Walkthrough

Each module has a **blue panel** (what a user experiences) and a **grey panel** (how it is built).

5.1 Authentication, Roles & Account Status

WHAT USERS SEE

- Sign in with Employee ID (or e-mail) and password.
- First sign-in creates a pending role request and sends a welcome e-mail.
- Suspended or inactive accounts are refused with a clear message.
- Admins see an Admin Console tab for role requests.

HOW IT WORKS

- `AuthService.LoginAsync` looks the user up, auto-syncing from EmployeeHub if unknown.
- Password checked against the local BCrypt hash, falling back to the MPO OIDC token endpoint.
- A JWT carrying the role claims is issued (default 480 min). `/api/auth/me` returns the current user.
- `[Authorize(Roles = …)]` attributes enforce RBAC.

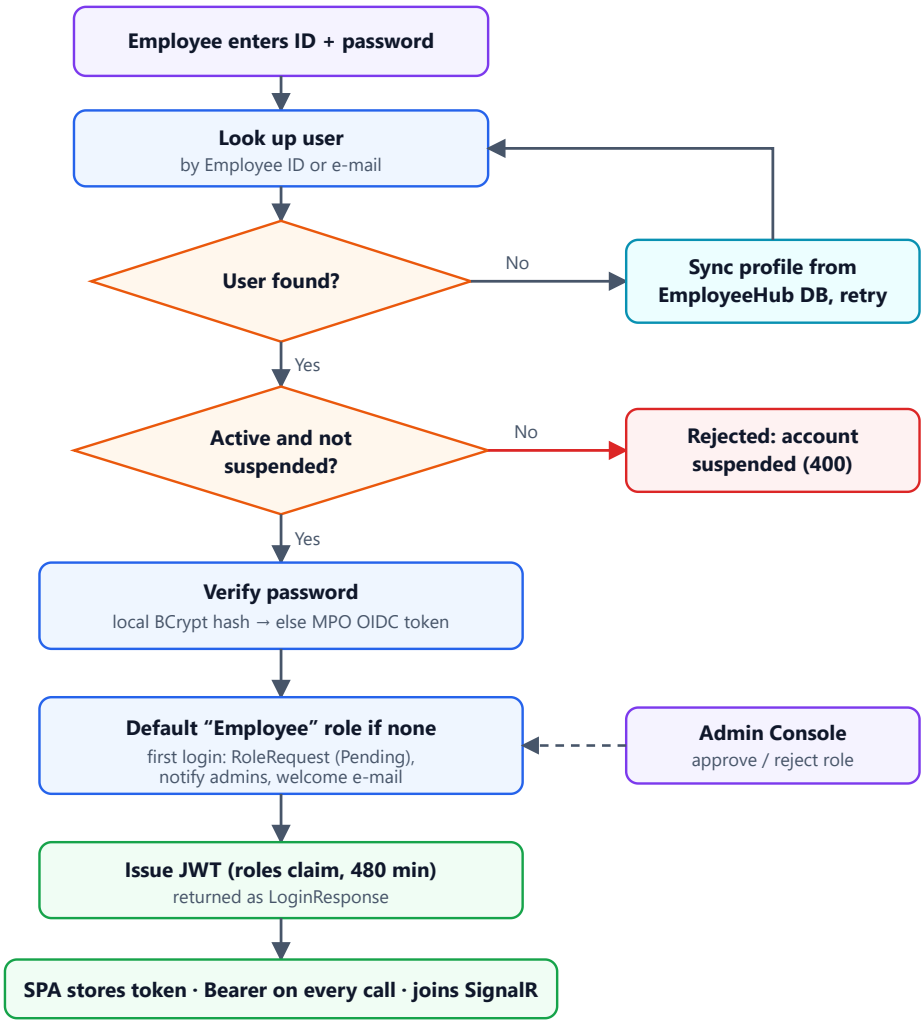


Figure 3 — Sign-in, first-login role request and token issue.

5.2 Newsfeed & Posts

WHAT USERS SEE

- A dashboard feed mixing posts, articles, videos and podcasts, with a "hot" ranking view.
- Quick posts (up to 400 characters) with images, documents and @mentions; images appear as a grid.
- Schedule a post to publish later; saved drafts.
- Text is screened against the restricted-keyword list before publishing.

HOW IT WORKS

- `/api/posts` (CRUD, "my", per-user) and `/api/feed` (`home` , `hot` , `dashboard`).
- Post fields include `AudienceType` , `Status` , `ScheduledDate` , `PublishedDate` ; attachments in `PostAttachments` .
- `ScheduledPostHostedService` publishes due posts and notifies followers.
- Hot score ≈ (Reactions + Comments×2 + Shares×3) ÷ time-decay, cached in `HotPostsScoreCache` .
- Mentions create `Mention` notifications.

5.3 Social Engagement

WHAT USERS SEE

- React to posts with **Like, Celebrate, Support or Heart**.
- Comment, reply to comments (nested), edit and delete own comments.
- Bookmark content to a “Saved” page; share inside Knome; report inappropriate content.

HOW IT WORKS

- `/api/interactions/{contentType}/{contentId}/` — one generic API for posts, articles, videos, podcasts, wikis and clips: `comments`, `react`, `reactions`, `share`, `bookmark`, `report`, `summary`.
- Reports land in a moderation queue (`reports`, `reports/pending`, `reports/{id}/resolve`).
- Every action publishes a notification and may award Karma.

5.4 Enterprise Communities

WHAT USERS SEE

- Browse public and private communities by category; request to join private ones.
- Community feed, member list, rules and FAQ, and a shared files library.
- Admins approve or reject join requests, add/remove members, promote admins and pin posts.

HOW IT WORKS

- `Community` (type, `ApprovalStatus`, rules, FAQ) and `CommunityMember` (`MemberType`, `Status`, decision audit fields).
- Endpoints: `join`, `leave`, `members`, `members/{id}/decide`, `admins/{id}`, `posts`, `posts/{id}/pin`, `files`.
- Safeguards: a community cannot lose its last admin; at most three pinned posts.
- Each decision notifies the user (`CommunityJoin`, `CommunityInvite`).

5.5 Direct Messaging (connection-gated & encrypted)

WHAT USERS SEE

- One-to-one chat with live delivery, typing indicators, online status and read receipts.
- Attachments, emoji picker, replies, edit, delete-for-me / delete-for-everyone, and six emoji reactions (👍 ❤️ 😂 😮 😢 🎉).
- **Only connected colleagues can chat.** For anyone else the message box is replaced by a lock banner with a **Connect** button (or “Request Pending”).
- The “Message” button on profiles and community members appears only for connected people.

HOW IT WORKS

- `/api/messages` : `users` , `conversations` , `history/{id}` , `send` , `{id}` (edit/delete), `{id}/reactions` , `read/{id}` , `online-users` .
- `SendMessageAsync` enforces the rule server-side via `AreUsersConnectedAsync` (accepted request *or* mutual follow).
- Message text is encrypted with AES-256-GCM before it is stored in `UserMessages` .
- Delivery uses SignalR groups named `User_{id}` ; an administrator can disable messaging globally.

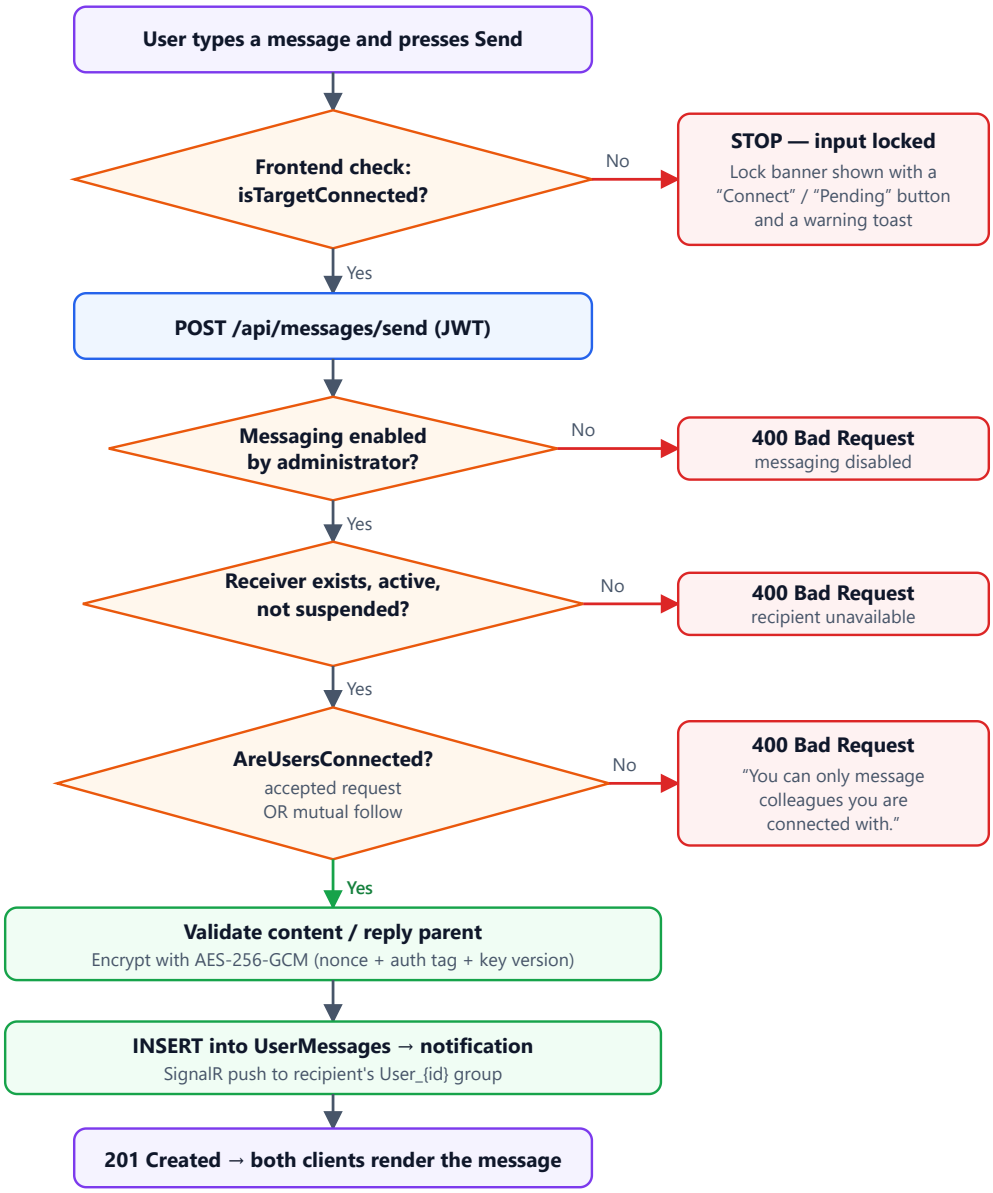


Figure 4 — Messaging is protected twice: in the UI and again on the server.

5.6 Network & Connections

WHAT USERS SEE

- Discover colleagues with mutual-connection counts and common communities.
- **Follow** (one-way, to see someone's content) or **Connect** (two-way relationship that unlocks chat).
- Send, accept, reject, cancel and remove connections; see pending requests.

HOW IT WORKS

- `ConnectionRequests` (`Pending` → `Connected`) plus two `Followers` rows are created on accept.
- Endpoints: `POST /users/{id}/connect` , `DELETE …/connect/cancel` , `POST /users/connect/accept|reject/{requestId}` , `DELETE …/connect` , `GET /users/connections/requests` , `GET /users/{id}/connections` .
- If both sides invite each other, the requests auto-merge into a connection.

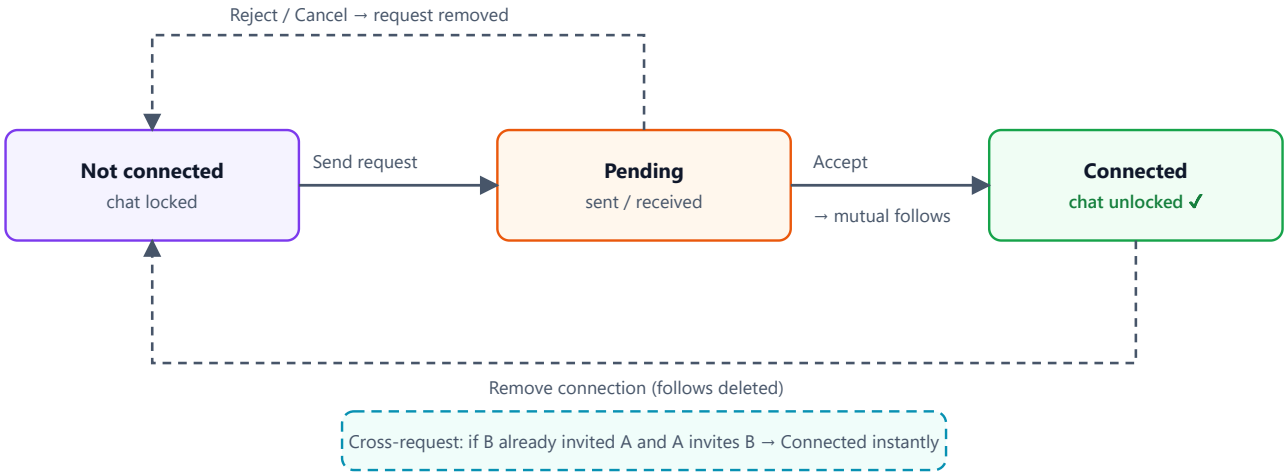


Figure 5 — Connection lifecycle. Messaging is allowed only in the "Connected" state.

5.7 Knowledge: Wikis & Articles

WHAT USERS SEE

- **Articles:** rich-text blog posts with tags, attachments, read-time and view counts.
- **Wikis:** structured knowledge pages made of reorderable sections, with collaborators and sharing.
- Version history with the option to restore an older version; archive and restore wikis.

HOW IT WORKS

- `/api/articles` with `ArticleVersions` snapshots; `/api/wikis` with `WikiSections` , `WikiCollaborators` , `WikiShares` , `WikiTags` , `WikiVersions` .
- Wiki endpoints include `sections` , `sections/reorder` , `collaborators` , `shares` , `versions/{id}/restore` , `activities` , `archive` , `restore` .
- Sharing and collaboration produce `Share` / `Community` (wiki) notifications.

5.8 Media Hub: Clips, Videos & Podcasts

WHAT USERS SEE

- **Clips:** short vertical videos with reactions, bookmarks, shares and viewer analytics.
- **Videos:** upload (up to 500 MB) or embed; tags; playlists support.
- **Podcasts:** audio series and episodes (up to 100 MB) with an audio player.

HOW IT WORKS

- `/api/clips`, `/api/videos`, `/api/podcasts`, plus `/api/media` for uploads (magic-byte file validation).
- Clips expose `viewers`, `engagement`, and admin moderation (`admin/all`, `admin/{id}/status`).
- Limits come from `MediaConstants`; feature switches can disable each module.

5.9 Karma & Gamification

WHAT USERS SEE

- Points for contributing and engaging; a tier badge and progress bar on the profile.
- Karma history page and a company leaderboard.

HOW IT WORKS

- `KarmaService` writes `KarmaTransaction` rows and updates `KarmaBalance` (`TotalPoints`, `BadgeLevel`); balances never go below zero.
- Daily caps prevent farming; `/api/karma`: `my`, `user/{id}`, `leaderboard`, `award`.

Activity	Points	Daily cap
Create a post	2	50
Publish an article	10	30
Upload a video / podcast	8 each	24 each
Receive a like / reaction	1	—
Receive a comment / share	2 / 3	—
Community participation	5	5
Add a comment / like / share	2 / 1 / 2	—

Tiers: Level 1 · 0–99 | Bronze · 100–499 | Silver · 500–999 | Gold · 1,000–4,999 | Platinum · 5,000+

5.10 Notification Center

WHAT USERS SEE

- A bell with an unread count; a list with All / Unread filters, mark-all-read and delete.
- A pop-up toast and a soft chime when something happens in real time.
- Clicking an alert opens the exact item (post, profile, community, wiki, connection requests …).
- A settings dialog to turn categories on or off (comments, reactions, followers, invites, mentions, community posts, jobs).

HOW IT WORKS

- Any service calls `INotificationService.PublishAsync(...)`; `PublishBroadcastAsync` handles HR announcements.
- Preferences are checked first, the row is saved in `Notifications`, the DTO is enriched (title, sender, `TargetUrl`) and pushed with `ReceiveNotification` to group `User_{id}`.
- Event types: Comment, Reaction, Mention, Follower, ConnectionRequest, Community, CommunityJoin, CommunityInvite, Share, Job, Badge, Wiki, HRAnnouncement.

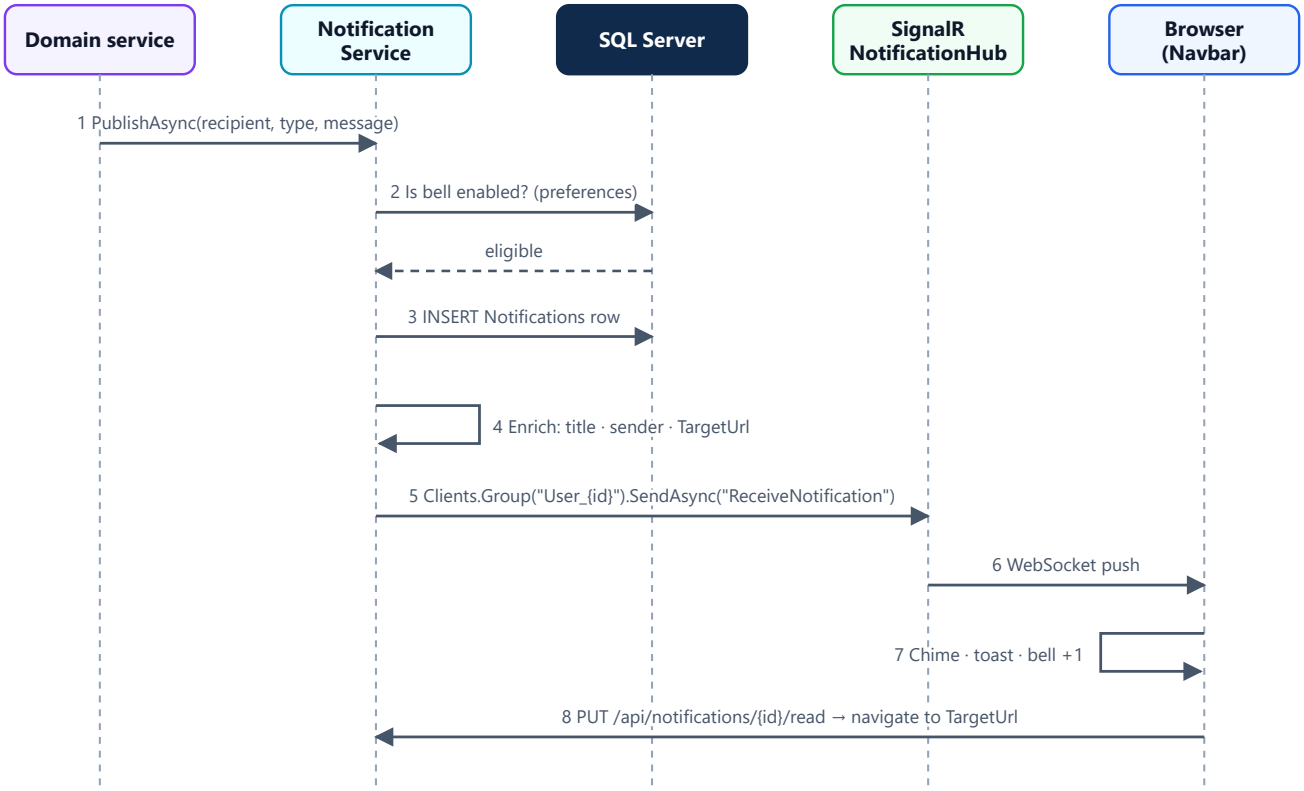


Figure 6 — Life of a notification, from the triggering event to the user's screen.

5.11 Admin Console & Governance

WHAT USERS SEE (ADMINS)

- Switches to turn modules on or off: messaging, e-mail, posts, articles, clips, wiki, videos, podcasts.
- Restricted-keyword list, moderation report queue, user suspension, role-request approvals, HR broadcast composer and analytics.

HOW IT WORKS

- `/api/settings` + `SystemSetting` entity, read by the frontend through `systemConfig.js`; controllers also check flags (e.g. messaging).
- `/api/interactions/restricted-keywords`, `/api/audit` (immutable `AuditLog`, System Admin only), `/api/analytics`.
- Suspensions use `IsPermanentlySuspended` / `SuspendedUntil` and are enforced at login and by a suspension guard.

5.12 Search, Jobs & Abbreviations

- **Global search** (`/api/search`) spans people, communities, posts, articles, videos, podcasts and jobs, with search history.
- **Jobs board** (`/api/jobs`) — HR/System Admin publish internal vacancies; `JobExpiryHostedService` closes expired postings automatically.
- **Abbreviations** (`/api/abbreviations`) — a shared company glossary.
- **Data archival** — `DataArchivalHostedService` plus `ARCHIVE_*.ps1` scripts keep logs and uploads from growing without limit.

6. Data Models & Database Overview

The `Knome` database holds 50+ entities. The diagram shows the tables most relevant to networking, messaging, communities and notifications. **PK** = primary key, **FK** = foreign key.

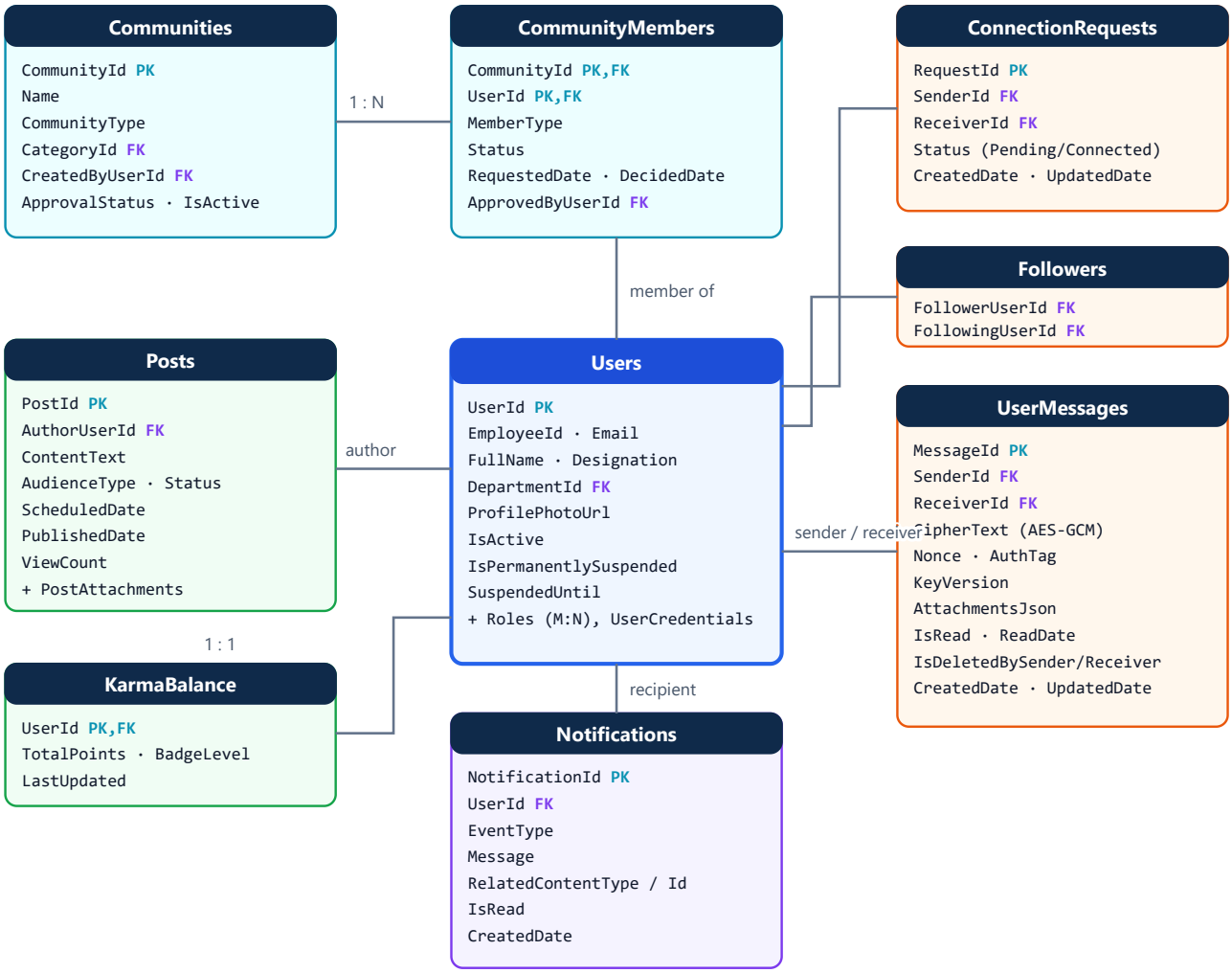


Figure 7 — Simplified entity-relationship diagram (key tables only).

6.1 Other important tables

Group	Tables
Identity & org	`Users`, `UserCredentials`, `Roles`, `RoleRequests`, `Departments`, `UserSkills`, `UserInterests`
Content	`Posts`, `PostAttachments`, `Articles` (+ `ArticleTags`, `ArticleVersions`, `ArticleAttachments`), `Videos` (+ `VideoTags`), `Podcasts`, `PodcastSeries`, `Clips`, `ClipShares`, `Wikis` (+ sections, collaborators, shares, tags, versions)
Engagement	`Comments`, `Reactions`, `Shares`, `Bookmarks`, `ContentViews`, `HotPostsScoreCache`, `ModerationReports`, `BlockedUrls`, `RestrictedKeywords`

Group	Tables
Communities	`Communities`, `CommunityMembers`, `CommunityPosts`, `Categories`
Social & chat	`Followers`, `ConnectionRequests`, `UserMessages`, `UserMessageReactions`
Gamification & alerts	`KarmaBalance`, `KarmaTransaction`, `Notifications`, `NotificationPreferences`
Governance & misc	`AuditLog`, `SystemSettings`, `SearchHistory`, `Jobs`, `Abbreviations`

7. Security, Privacy & Encryption

7.1 Authentication & authorization

- **JWT Bearer (HS256).** The signing secret comes from configuration or the `KNOME_JWT_SECRET` environment variable and must be at least 32 characters. Default lifetime is 480 minutes (`JwtSettings.ExpiryMinutes`). The API also accepts MPO-issued tokens through a second authentication scheme.
- **Passwords** are stored as BCrypt hashes (`UserCredentials.PasswordHash`).
- **RBAC.** Four roles are defined in `Constants/Roles.cs`; endpoints use `[Authorize(Roles = …)]` and named policies.
- **Brute-force protection.** `LoginRateLimiter` allows 30 login requests per minute per client IP, then returns HTTP 429.
- **Suspension.** Suspended or inactive users cannot sign in; suspension guards protect write operations.

7.2 Direct-message encryption (AES-256-GCM)

Message bodies are never stored as plain text. `MessageEncryptionService` encrypts each message with AES-256 in Galois/Counter Mode, which provides both **confidentiality** and **tamper detection**.

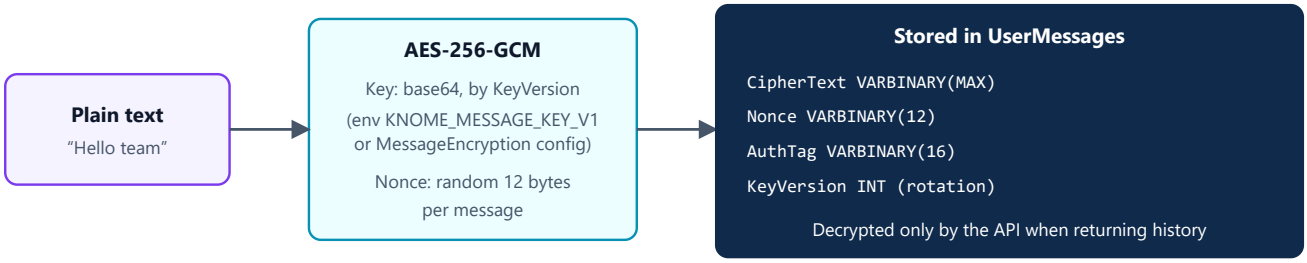


Figure 8 — Encryption at rest for direct messages.

> **Important to understand**
>
> Encryption is applied **by the server** (encryption at rest). It protects the database contents, but it is not client-side end-to-end encryption — the API can decrypt messages. Keep the message keys in environment variables or a secrets manager, never in source control.

7.3 Connection-gating logic

Two users may exchange direct messages only when `AreUsersConnectedAsync(a, b)` returns true:

```
public async Task<bool> AreUsersConnectedAsync(int userId1, int userId2)
{
    // 1) An accepted / connected ConnectionRequest in either direction
    // 2) OR a MUTUAL follow: user1 follows user2 AND user2 follows user1
    // A one-way follow is NOT enough.
}
```

- **Server enforcement:** `SendMessageAsync` throws `BadRequestException("You can only message colleagues you are connected with.")` when the check fails — a custom client cannot bypass the UI.
- **UI enforcement:** `Messages.jsx` locks the composer, shows a banner and offers **Connect**; Profile and Community pages show the Message button only to connected users.
- **Conversation list & search** expose an `IsConnected` flag so badges ("Connected" / "Not Connected") are accurate.

7.4 Other safeguards

Control	Detail
Security headers	`SecurityHeadersMiddleware` : HSTS, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.
File upload safety	Magic-byte (file signature) validation, size caps (video 500 MB, podcast 100 MB, request limit 500 MB).
Input validation	FluentValidation on DTOs; HTML sanitised in the frontend (`sanitizeHtml.js`); restricted-keyword and blocked-URL screening.
Log privacy	`PiiScrubbingEnricher` removes sensitive values from Serilog events.
Auditability	Immutable `AuditLog` of administrative actions, readable only by System Administrators.
Error hygiene	`ExceptionHandlingMiddleware` maps exceptions to 400/403/404/500 without leaking stack traces.

7.5 Known risks to review before production

Findings from the code review — recommended hardening

- **Development login shortcuts.** `AuthService.LoginAsync` accepts the literal passwords `Password@123` and `SSO_BYPASS` for any account, creates a default credential if one is missing, and contains hard-coded employee-ID special cases. **Remove these before any production or shared deployment.**
- **Credentials in scripts.** `START_KNOME.ps1` embeds a SQL login and server names. Move them to environment variables or user secrets.
- **SignalR hub trust.** `NotificationHub.JoinUserGroup` only rejects a mismatch when the connection carries a user id, and `SendDirectMessage` relays a client-supplied payload to the recipient group without the connection check. Verify the hub requires authentication and consider applying the same connectivity rule there.
- **Server-side encryption keys.** Ensure production keys are supplied through environment variables and rotated via `KeyVersion`.

8. API Reference

All endpoints are under `/api` and return `{ "success": bool, "message": string, "data": … }`. Send `Authorization: Bearer <token>` on every call except login. Swagger UI lists every route interactively when the API runs in Development.

Authentication — `/api/auth`

Method	Path	Purpose
POST	`/auth/login`	Sign in (rate limited) → token + user
POST	`/auth/logout`	Sign out
GET	`/auth/me`	Current user profile and roles

Users, Network & Roles — `/api/users`

Method	Path	Purpose
GET / PUT	`/users/profile`	Read / update own profile (also `profile/bio`, `profile/skills`, POST `profile/image`)
GET	`/users`, `/users/{id}`, `/users/suggestions`	Directory, a profile, suggested colleagues
POST / DELETE	`/users/{id}/follow`	Follow / unfollow
GET	`/users/{id}/followers`, `/following`, `/connections`	Network lists
POST	`/users/{id}/connect`	Send connection request
DELETE	`/users/{id}/connect/cancel`, `/users/{id}/connect`	Cancel request / remove connection
POST	`/users/connect/accept/{requestId}`, `/connect/reject/{requestId}`	Respond to a request
GET	`/users/connections/requests`	Pending received & sent requests
PUT	`/users/{id}/department`, `/roles`, `/activate`, `/suspend`	Admin user management
GET / POST	`/users/role-requests`, `/{id}/approve`, `/{id}/reject`	Role-request workflow (HR / System Admin)

Posts & Feed — `/api/posts`, `/api/feed`

Method	Path	Purpose
GET / POST	`/posts`	List / create posts
GET	`/posts/my`, `/posts/user/{userId}`, `/posts/{postId}`	Own, by author, single post
PUT / DELETE	`/posts/{postId}`	Edit / delete
POST	`/posts/{postId}/view`	Record a view
GET	`/feed/home`, `/feed/hot`, `/feed/dashboard`	Personalised, ranked and dashboard feeds

Interactions — `/api/interactions`

Method	Path	Purpose
GET / POST	`/interactions/{contentType}/{contentId}/comments`	List / add comments
PUT / DELETE	`/interactions/comments/{commentId}`	Edit / delete a comment
POST / GET	`…/{contentId}/react`, `/reactions`, `/reactions/list`	React and read reactions
POST	`…/{contentId}/share`, `/bookmark`, `/report`	Share, bookmark, report
GET	`/interactions/bookmarks/my`, `/saved-content`, `/saved-content/count`	Saved items
GET / PUT	`/interactions/reports`, `/reports/pending`, `/reports/{id}/resolve`	Moderation queue
GET / POST	`/interactions/restricted-keywords`, `/validate-security`	Keyword list and content screening

Communities — `/api/communities`

Method	Path	Purpose
GET	`/communities`, `/my`, `/user/{id}`, `/categories`, `/check-name`, `/pending`	Browse and lookups
POST / PUT / DELETE	`/communities`, `/{id}`	Create, edit, delete
POST	`/communities/{id}/approve`, `/reject`	Approve / reject a new community
POST	`/communities/{id}/join`, `/leave`	Membership

Method	Path	Purpose
GET / POST / PUT / DELETE	`/communities/{id}/members`, `/members/{userId}/decide`, `/members/{userId}`	Member administration
POST / DELETE	`/communities/{id}/admins/{userId}`	Grant / revoke admin
GET / POST / PUT	`/communities/{id}/posts`, `/posts/{postId}/pin`	Community feed and pinning
GET / POST / DELETE	`/communities/{id}/files`	Shared file library

Messaging — `/api/messages`

Method	Path	Purpose
GET	`/messages/users?query=`	Search colleagues (includes `isConnected`)
GET	`/messages/conversations`	Conversation list with previews & unread counts
GET	`/messages/history/{otherUserId}`	Paged, decrypted history
POST	`/messages/send`	Send (connection-gated, encrypted)
PUT / DELETE	`/messages/{messageId}`	Edit / delete (`?deleteForEveryone=true`)
POST	`/messages/{messageId}/reactions`	Toggle an emoji reaction
POST	`/messages/read/{otherUserId}`	Mark conversation as read
GET	`/messages/online-users`	Currently online user ids

Notifications — `/api/notifications`

Method	Path	Purpose
GET	`/notifications`, `/unread-count`	List (paged, `unreadOnly`) and badge count
PUT / POST	`/notifications/{id}/read`, `/read-all`	Mark read
DELETE	`/notifications/{id}`	Delete
GET / PUT	`/notifications/preferences`	Per-event bell / e-mail preferences
GET / POST / PUT / DELETE	`/notifications/broadcasts`, `/broadcast`, `/broadcast/{id}`	HR announcements (HR / System Admin)

Knowledge, Media, Karma & Admin

Base route	Highlights
`/api/wikis`	CRUD, `my`, `shared`, `recent`, `tags`, `{id}/sections`, `/collaborators`, `/shares`, `/versions`, `/restore`, `/archive`, `/activities`
`/api/articles`	CRUD, version history
`/api/videos`, `/api/podcasts`	CRUD, series / episodes, "my" lists
`/api/clips`	CRUD, `{id}/view`, `/react`, `/bookmark`, `/share`, `/viewers`, `/engagement`, `me`, `admin/all`, `admin/{id}/status`
`/api/media`	File and image uploads
`/api/karma`	`my`, `user/{id}`, `leaderboard`, `award`
`/api/search`	Unified search and history
`/api/jobs`	Filtered job listings, CRUD for HR / System Admin
`/api/settings`	Platform feature switches and approval settings
`/api/audit`, `/api/analytics`	Audit log (System Admin), HR analytics
`/api/abbreviations`	Company glossary
SignalR `/hubs/notifications`	Server → client events: `ReceiveNotification`, `ReceiveDirectMessage`, `UserTyping`, `UserPresenceChanged`. Client → server: `JoinUserGroup`, `LeaveUserGroup`, `SendTyping`, `GetOnlineUsers`.

9. Local Setup & Developer Onboarding

9.1 Prerequisites

Tool	Version / notes
.NET SDK	**.NET 10** (project targets `net10.0`)
Node.js + npm	A current LTS release (Vite 8 needs a modern Node; 22 LTS recommended)
SQL Server	2022+ (Express or Developer edition is enough), with a database named `Knome`
Windows + PowerShell	Required by the one-click launcher and the `SETUP_*.ps1` scripts
Optional	IIS with the ASP.NET Core Hosting Bundle (for ports 8080/8081), Git, SSMS

9.2 Step-by-step

Step 1 — Get the code

```
git clone <your-repository-url> "Knome main"
cd "Knome main"
```

Step 2 — Create and prepare the database

1. Create an empty SQL Server database called `Knome` and restore/provision the core schema (see `Documentation/Database` and `Documentation/Architecture`).
2. Run the helper scripts for feature tables (the launcher also runs them automatically): `SETUP_WIKI_DB.ps1`, `SETUP_CLIPS_DB.ps1`, `SETUP_MESSAGES_DB.ps1` (or `CreateUserMessages.sql` in SSMS).
3. Optionally load sample users with the seed script referenced in `README.md` (`Documentation/Architecture/Seed_Test_Credentials.sql`).

Step 3 — Configure the backend

Edit `Backend/Knome.API/appsettings.Development.json` (never commit real secrets) or use environment variables / user-secrets.

```
{
  "ConnectionStrings": { "DefaultConnection": "Server=YOUR_SERVER;Database=Knome;User
Id=YOUR_USER;Password=YOUR_PASSWORD;TrustServerCertificate=True" },
  "Jwt": { "ExpiryMinutes": 480 },
  "MessageEncryption": { "DefaultKeyVersion": 1 }
}
# Environment variables (recommended for secrets)
KNOME_JWT_SECRET        = <random string, 32+ characters>
KNOME_MESSAGE_KEY_V1    = <base64 of 32 random bytes>
```

> **Tip**
>
> Generate a message key in PowerShell: `[Convert]::ToBase64String((1..32 | % { Get-Random -Max 256 }) -as [byte[]])`. Check the exact configuration key names in `appsettings.json` for your branch.

Step 4 — Run the backend

```
cd Backend/Knome.API
dotnet restore
dotnet run --launch-profile http        # API on http://localhost:5096  (Swagger at /swagger)
```

Step 5 — Run the frontend

```
cd knomeUI/frontend
npm install
npm run dev                             # UI on http://localhost:5173 (proxies /api → :5096)
```

Step 6 — One-click alternative

From the repository root run `.\START_KNOME.bat`. The launcher frees ports 5095/5096/5173, runs `npm install` on first use, verifies the database, ensures wiki/clips/messages tables exist, starts the API and the UI, and (if IIS is configured) starts the `Knome` and `EmployeeHub` sites on 8080/8081. Press `Ctrl+C` to stop everything.

URL	What it is
`http://localhost:5173`	Knome development UI
`http://localhost:5096` (or 5095)	Knome REST API & SignalR hub
`http://localhost:8080` / `:8081`	IIS-hosted Knome portal / EmployeeHub (optional)

9.3 Typical developer workflow

1. Create a feature branch; keep controllers thin and put rules in services.
2. Add or change DTOs and FluentValidation rules together.
3. If the schema changes, update the SQL and re-scaffold the models — do not hand-edit `Models/` or `Data/KnomeDbContext.cs`.
4. Run `dotnet build` and `npm run build` (and `npm run lint`) before opening a pull request.
5. Follow the team rules in `AGENTS.md` / `CLAUDE.md`.

10. FAQ & Troubleshooting

Symptom	Likely cause	Fix
Chat says *"You can only message colleagues you are connected with."*	Users are not connected (or only one follows the other).	Send and accept a connection request, or ensure both users follow each other. Check `ConnectionRequests.Status` is `Connected`.
Message box is replaced by a lock banner	Intended behaviour for non-connected users.	Click **Connect**; once accepted the composer unlocks automatically.
"Direct messaging is currently disabled by system administrator."	The messaging feature switch is off.	A System Administrator enables it in the Admin Console (`EnableMessaging`).
Live updates stop; "Reconnecting to Knome messenger…"	SignalR WebSocket dropped (network, sleep, API restart, wrong host).	Check the API is running; the client auto-reconnects. Verify the hub URL `http://<host>:5096/hubs/notifications` and that CORS allows your UI origin.
401 Unauthorized on every call	Missing/expired token, or JWT secret changed.	Sign in again; ensure `KNOME_JWT_SECRET` is set and identical across restarts.
App fails at start: JWT secret error	Secret missing or under 32 characters.	Provide a longer `KNOME_JWT_SECRET`.
"Cryptographic key for version N is not configured."	No message-encryption key found.	Set `KNOME_MESSAGE_KEY_V1` (base64, 32 bytes) or the `MessageEncryption` config section.
Cannot connect to database	Wrong server/credentials, SQL Server stopped, or TCP/IP disabled.	Test the connection string in SSMS; enable TCP/IP; use `TrustServerCertificate=True` for local certificates; check the launcher's server-name list.
Invalid object name 'UserMessages' / wiki / clips table	Setup scripts not run.	Run `SETUP_MESSAGES_DB.ps1`, `SETUP_WIKI_DB.ps1`, `SETUP_CLIPS_DB.ps1`.
Port already in use (5096 / 5173)	An old process is still running.	Re-run `START_KNOME.bat` (it frees the ports) or stop the process manually.
No notification appears	User disabled that category, or the event was triggered by themselves.	Open notification settings and re-enable the category; self-triggered events are ignored by design.
HTTP 429 on login	More than 30 attempts per minute from one IP.	Wait one minute.

Symptom	Likely cause	Fix
Upload rejected	File too large or type/signature mismatch.	Respect limits (video 500 MB, podcast 100 MB) and use genuine file types.
“Your account is suspended by system admin.”	Account inactive or suspended.	Ask an HR/System Administrator to activate the account.

Frequently asked questions

What is the difference between Follow and Connect? Follow is one-way (you see their content). Connect needs the other person’s acceptance and creates a two-way relationship — only that unlocks private chat.

Can administrators read private messages? Messages are encrypted at rest in the database, but the API decrypts them for participants. Treat access to the server and keys as sensitive.

Where do new roles come from? Everyone starts as Employee. A role request is created at first sign-in; HR or System Administrators approve elevated roles in the Admin Console.

How do I add a new notification type? Add a constant in `NotificationTypes`, call `PublishAsync` from the relevant service, and (optionally) add a title/target mapping in `NotificationService.EnrichNotificationDtoAsync`.

Appendix — Glossary & Feature Status

A.1 Glossary

Term	Meaning
BFF	Backend-for-Frontend: a small server that adapts the API for one UI.
Connection	An accepted, two-way relationship between two employees (not the same as Follow).
DTO	Data Transfer Object: the JSON shape sent to / from the API.
EF Core	Entity Framework Core, the .NET object-relational mapper.
JWT	JSON Web Token: signed proof of identity and roles sent on each request.
Karma	Points earned through participation; maps to tiers (Bronze … Platinum).
RBAC	Role-Based Access Control: permissions depend on the user's role.
SignalR	ASP.NET real-time library using WebSockets.
YARP	"Yet Another Reverse Proxy" from Microsoft, used by the optional gateway.

A.2 Feature status

Capability	Status	Evidence / notes
JWT auth, RBAC, role-request workflow	**Implemented**	`AuthService`, `UserController`
Posts, scheduling, @mentions, restricted words	**Implemented**	`PostService`, `ScheduledPostHostedService`
Reactions, nested comments, bookmarks, shares, reports	**Implemented**	`InteractionController`
Communities with approvals and file library	**Implemented**	`CommunityController`
Encrypted 1-to-1 chat, connection-gated	**Implemented**	`MessagesController`, `MessageEncryptionService`
Connections & follows	**Implemented**	`UserService`, `Network.jsx`
Wikis (sections, versions, collaborators), Articles	**Implemented**	`WikiController`, `ArticleController`
Clips, videos, podcasts	**Implemented**	`ClipsController`, `VideoController`, `PodcastController`
Karma, leaderboard, tiers	**Implemented**	`KarmaService`, `karmaEngine.js`
Notification center with preferences and toasts	**Implemented**	`NotificationService`, `Navbar.jsx`

Capability	Status	Evidence / notes
Admin Console, feature switches, audit log	**Implemented**	`AdminConsole.jsx`, `SystemSettingsController`, `AuditLogController`
WebRTC audio / video calling	Not found	No WebRTC / signalling code in backend or frontend. Treat as roadmap.
Polls in posts	Not found	No poll entity, endpoint or UI found. Treat as roadmap.
Hashtag discovery	**Unverified**	Tags exist for articles, videos and wikis; dedicated hashtag search was not confirmed.

Keeping this document current

Update the matrix, API tables and feature status whenever a controller, role or module changes. Regenerate the PDF from this HTML file after each update.